

Chap 14 File

https://github.com/kkh3363/C_Lang

Contents

01 File Create

02 Write to Files

03 Read Files

01 Create File

■ File Handling

- In C, you can create, open, read, and write to files by declaring a pointer of type FILE, and use the fopen() function:

```
FILE *fptr;  
fptr = fopen(filename, mode);
```

- FILE is basically a data type, and we need to create a pointer variable to work with it (fptr).
- To open a file, use the fopen() function, which takes two parameters:

Files

https://github.com/kkh3363/C_Lang

```
FILE *fptr;  
fptr = fopen(filename, mode);
```

Parameter	Description
<i>filename</i>	The name of the actual file you want to open (or create), like filename.txt
<i>mode</i>	A single character, which represents what you want to do with the file (read, write or append): w - Writes to a file a - Appends new data to a file r - Reads from a file

File

https://github.com/kkh3363/C_Lang

■ mode

Mode	Description
"w"	We use this mode to open or create a text file in writing mode. The data existing in the file is erased.
"r"	We use this mode for opening an existing file in reading mode. The file to be opened must exist.
"a"	We use this mode to open a text file in append mode. The existing data of the file is not erased as in "w" mode.
"w+"	We use this mode to open a text file in both reading and writing mode.
"a+"	We use this mode to open a text file in both reading and writing mode.
"r+"	We use this mode to open a text file in both reading and writing mode.
"wb"	We use this mode to open or create a binary file in writing mode.
"rb"	We use this mode to open a binary file in reading mode.
"ab"	We use this mode to open a binary file in append mode.
"wb+"	We use this mode to open a binary file in both reading and writing modes.
"rb+"	We use this mode to open a binary file in both reading and writing modes.
"ab+"	We use this mode to open a binary file in both reading and writing modes.

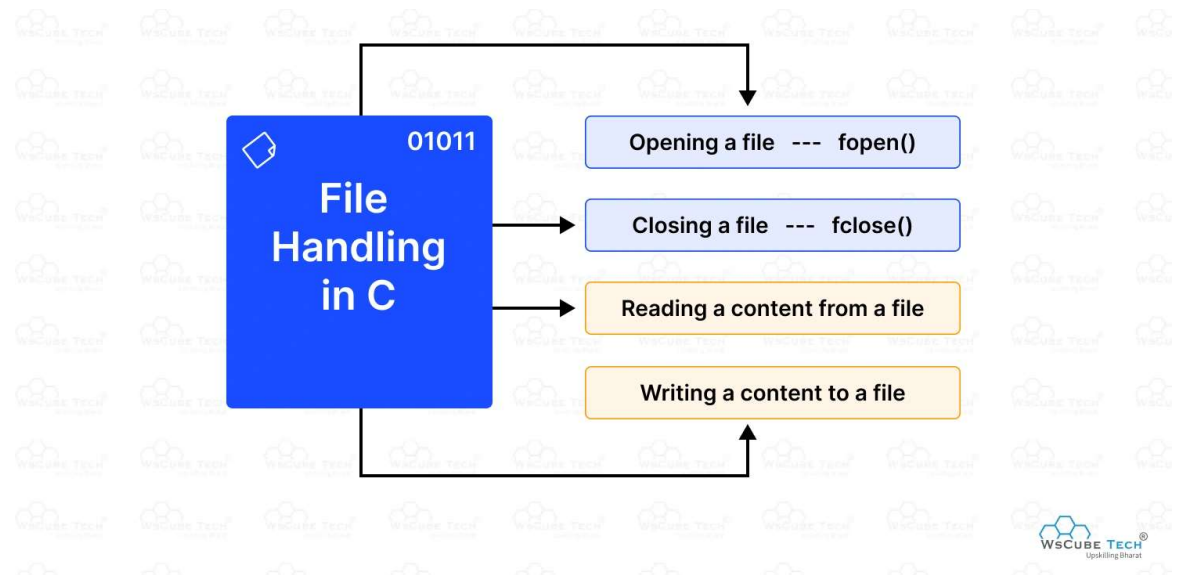
Files

https://github.com/kkh3363/C_Lang

■ Create a File

- To create a file, you can use the **w** mode inside the `fopen()` function.
- The `w` mode is used to write to a file. However, if the file does not exist, it will create one for you:

```
FILE *fp;  
  
// Create a file  
fp = fopen("filename.txt", "w");  
  
// Close the file  
fclose(fp);
```



■ Write To a File

- The w mode means that the file is opened for writing. To insert content to it, you can use the fprintf() function and add the pointer variable (fp in our example) and some text:

```
#include <stdio.h>

int main() {
    FILE *fp;

    // Open a file in writing mode
    fp = fopen("test.txt", "w");

    // Write some text to the file
    fprintf(fp, "Some text");

    // Close the file
    fclose(fp);

    return 0;
}
```


- Append Content To a File
 - If you want to add content to a file without deleting the old content, you can use the a mode.
 - The a mode appends content at the end of the file:

```
#include <stdio.h>

int main() {
    FILE *fp;

    // Open a file in writing mode
    fp = fopen("test.txt", "a");

    // Write some text to the file
    fprintf(fp, "\nHi everybody");

    // Close the file
    fclose(fp);

    return 0;
}
```

Files

https://github.com/kkh3363/C_Lang

■ Read a File

- To read from a file, you can use the **r** mode:
- if file not exists, then fopen() return null.

```
fp = fopen("filename.txt", "r");  
if ( fp == NULL )  
    printf("File not exists.");
```

```
if ( (fp = fopen("filename.txt", "r")) == NULL )  
    printf("File not exists.");
```

```
#include <stdio.h>
```

```
int main() {  
    FILE *fp;
```

```
    // Open a file in read mode  
    fp = fopen("filename.txt", "r");
```

```
    // Store the content of the file  
    char myString[100];
```

```
    // Read the content and store it inside myString  
    fgets(myString, 100, fp);
```

```
    printf("%s", myString); // Print file content
```

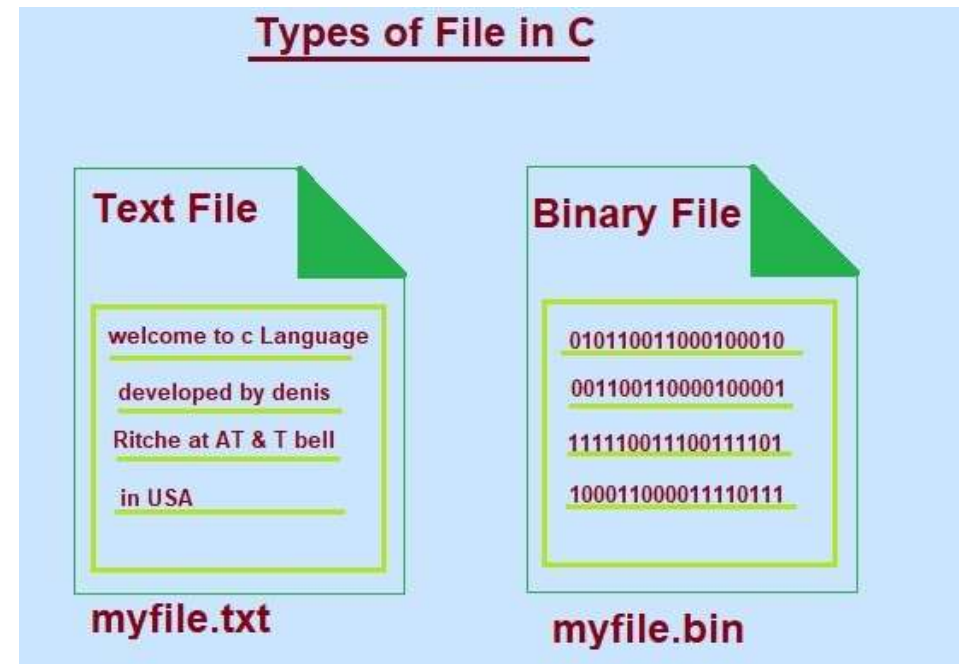
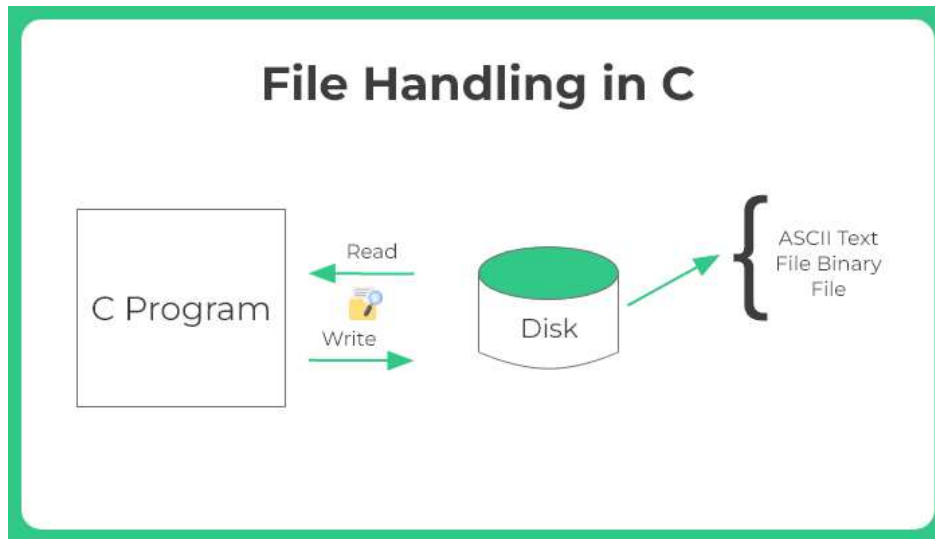
```
    fclose(fp); // Close the file
```

```
    return 0;  
}
```

Files

https://github.com/kkh3363/C_Lang

■ Read a File



■ File Function

function	description.	mode
fprintf(fout, format, param)	write to file	Text
fscanf(fin, format, param)	read to file	Text
fputc(c, fout), putc(c, fout)	write to file (character)	Text
fgetc(fin), getc(fin)	read to file (character)	Text
fputs(s, fout)	write to file (string)	Text
fgets(s, n, fin)	read to file (string)	Text
fwrite(buf, size, n, fout)	write to file (binary block)	Binary
fread(buf, size, n, fin)	read to file (binary block)	Binary

■ Example Code

■ file copy

[Code ref](#)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main(int argc, char* argv[])
{
    FILE* fpr ;
    FILE* fpw;
    char file;

    if (argc != 3)
    {
        printf("usage %s : ", argv[0]);
        printf("copyfile [source filename] [output filename]\n");
        return 0;
    }
}
```

```
if ( (fpr = fopen(argv[1], "rb")) == NULL )
    return 0;
    fpw = fopen(argv[2], "wb");

    while (!feof(fpr))
    {
        fread(&file, sizeof(char), 1, fpr);
        fwrite(&file, sizeof(char), 1, fpw);
    }

    fclose(fpr);
    fclose(fpw);

    return 0;
}
```

■ Parsing Text or Data in C

■ String to Data field

[Code ref](#)

```
#include <stdio.h>
#include <string.h>

int main() {
    char data[] = "apple,banana,cherry";
    char *token = strtok(data, ",");

    while (token != NULL) {
        printf("Fruit: %s\n", token);
        token = strtok(NULL, ",");
    }
    return 0;
}
```

■ Mini project

- phonebook
 - add function.
 - saveFile()
 - readFile()

[Code ref](#)

Thank You !